

Node.js Class Plan: From JavaScript Runtime to HTTP Server Internals

Class Title

Node.js Under the Hood: How Single-Threaded JavaScript Serves Many Web Requests

Target Students

Beginner to intermediate JavaScript students preparing for Node.js / Express.js interviews.

Duration

3 hours

1. Learning Outcomes

By the end of this class, students will be able to:

1. Explain what Node.js is and why it is not just “JavaScript outside the browser.”
 2. Clearly differentiate:
 3. Single-threaded vs multithreaded
 4. Synchronous vs asynchronous
 5. Blocking vs non-blocking
 6. Explain how Node.js handles many HTTP requests using:
 7. Call stack
 8. Event loop
 9. Callback queue
 10. OS-level I/O
 11. libuv worker pool
 12. Use CommonJS modules with `module.exports` and `require`.
 13. Understand npm, `package.json`, dependencies, dev dependencies, and scripts.
 14. Use Node.js built-in `url` and `http` modules.
 15. Create a basic HTTP server from scratch.
 16. Answer common Node.js interview questions confidently.
-

2. Class Opening: Big Question

Start the class with this question:

"If JavaScript is single-threaded, how can Node.js handle thousands of users at the same time?"

Tell students that the full class will answer this question practically.

Use this mental model:

JavaScript execution is single-threaded, but Node.js is not just JavaScript. Node.js uses V8, libuv, the event loop, OS-level async I/O, and a worker pool to handle many operations efficiently.

3. Topic Flow

Part 1: Running JavaScript in Node.js

Concepts

- Browser JavaScript vs Node.js JavaScript
- V8 engine
- Node.js runtime
- Running files using:

```
node app.js
```

Demo Create `app.js`:

```
console.log("Node.js is running JavaScript outside the browser");
console.log(__dirname);
console.log(__filename);
```

Interview Angle Possible question:

What is Node.js?

Expected answer:

Node.js is a JavaScript runtime built on the V8 engine that allows JavaScript to run outside the browser, commonly used for building servers, APIs, tools, and backend applications.

Part 2: Single-threaded vs Multithreaded

Concepts

- JavaScript code runs on a main thread.
- Node.js can still handle many users because it does not wait for slow I/O tasks.
- Some tasks are delegated to OS or worker pool.
- Network I/O, file I/O, timers, callbacks, and async tasks are coordinated through the event loop.

Classroom Visualization Draw this on board:

```
Client Requests
  |
  v
Node.js Server
  |
  v
Main JS Thread
  |
  +--> Fast JS execution
  +--> Async I/O delegated
  +--> Callback returns later through event loop
```

Interview Angle Possible question:

Is Node.js single-threaded?

Expected answer:

JavaScript execution in Node.js is single-threaded by default, but Node.js internally uses the event loop, OS-level async I/O, and a worker pool for some operations. So Node.js can handle concurrent I/O without creating one thread per request.

Part 3: Synchronous vs Asynchronous

Demo 1: Synchronous Code

```
console.log("1");
console.log("2");
console.log("3");
```

Output:

```
1  
2  
3
```

Demo 2: Asynchronous Code

```
console.log("1");  
  
setTimeout(() => {  
  console.log("2");  
}, 0);  
  
console.log("3");
```

Output:

```
1  
3  
2
```

Teaching Point Students should understand that asynchronous does not mean “faster always.” It means the task is scheduled and completed later without blocking the main flow.

Interview Angle Possible question:

Why does `setTimeout` with 0ms not execute immediately?

Expected answer:

Because the callback is scheduled and must wait until the current call stack is empty before the event loop can push it back for execution.

Part 4: Blocking vs Non-blocking

Demo 1: Blocking Example

```
const fs = require("fs");  
  
console.log("Start");  
  
const data = fs.readFileSync("demo.txt", "utf8");
```

```
console.log(data);  
console.log("End");
```

Demo 2: Non-blocking Example

```
const fs = require("fs");  
  
console.log("Start");  
  
fs.readFile("demo.txt", "utf8", (err, data) => {  
  if (err) throw err;  
  console.log(data);  
});  
  
console.log("End");
```

Teaching Point Blocking means the main thread waits. Non-blocking means Node.js continues execution and handles the result later.

Interview Angle Possible question:

What is the difference between asynchronous and non-blocking?

Expected answer:

Asynchronous describes the programming model where a result is handled later. Non-blocking describes runtime behavior where execution does not wait for a slow operation to finish.

Part 5: Event Loop Visualization

Use visual tools to show:

- Call stack
- Callback queue
- Event loop
- Timers
- Async callbacks

Class Activity Ask students to predict the output:

```
console.log("A");
```

```
setTimeout(() => {  
  console.log("B");  
}, 0);  
  
Promise.resolve().then(() => {  
  console.log("C");  
});  
  
console.log("D");
```

Expected output:

```
A  
D  
C  
B
```

Teaching Point Explain difference between:

- Synchronous stack
- Microtask queue
- Timer/callback queue

Part 6: What is a Module in Node.js?

Concepts

- A module is a reusable file or package.
- Every file in Node.js can be treated as a module.
- CommonJS uses `require`.
- Exported values are shared through `module.exports`.

Demo

`math.js`

```
function add(a, b) {  
  return a + b;  
}  
  
function multiply(a, b) {  
  return a * b;  
}
```

```
module.exports = {  
  add,  
  multiply  
};
```

app.js

```
const math = require("./math");  
  
console.log(math.add(5, 3));  
console.log(math.multiply(5, 3));
```

Interview Angle Possible question:

What is the difference between `exports` and `module.exports`?

Expected answer:

`exports` is a shortcut reference to `module.exports`. But if we reassign `exports` directly, it no longer points to the original exported object. For full control, use `module.exports`.

Part 7: NPM and package.json

Concepts

- npm is the package manager for Node.js.
- `package.json` describes project metadata, dependencies, scripts, version, entry file, and package configuration.
- `node_modules` stores installed packages.
- `package-lock.json` locks exact dependency versions.

Demo

```
npm init -y  
npm install express  
npm install nodemon --save-dev
```

Example `package.json`:

```
{  
  "name": "node-class-demo",
```

```
"version": "1.0.0",
"main": "server.js",
"scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js"
},
"dependencies": {
  "express": "^5.0.0"
},
"devDependencies": {
  "nodemon": "^3.0.0"
}
}
```

Interview Angle Possible question:

What is the difference between dependencies and devDependencies?

Expected answer:

Dependencies are required to run the application in production. devDependencies are used during development, testing, formatting, building, or local tooling.

Part 8: URL Module

Concepts

- Parse incoming request URLs.
- Extract pathname and query parameters.
- Understand routing basics before Express.

Demo

```
const { URL } = require("url");

const myUrl = new URL("http://localhost:3000/products?id=10&category=book");

console.log(myUrl.pathname);
console.log(myUrl.searchParams.get("id"));
console.log(myUrl.searchParams.get("category"));
```

Mini Exercise Students parse:


```
http://localhost:3000/users?role=admin&active=true
```

Expected:

- pathname: `/users`
- role: `admin`
- active: `true`

Part 9: HTTP Module and Create HTTP Server

Concepts

- Node.js can create a server without Express.
- `http.createServer()` receives request and response objects.
- Request contains URL, method, headers.
- Response sends status, headers, and body.

Demo

```
const http = require("http");

const server = http.createServer((req, res) => {
  console.log("Request received");
  console.log("Method:", req.method);
  console.log("URL:", req.url);

  res.writeHead(200, { "Content-Type": "text/plain" });
  res.end("Hello from Node.js HTTP server");
});

server.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

Browser Test Open:

```
http://localhost:3000
```

Terminal Test

```
curl http://localhost:3000
```

Part 10: Server Internals Demonstration

Show what happens when a request comes in:

```
Browser / Client
  |
  v
HTTP Request
  |
  v
Node.js HTTP Server
  |
  v
Callback enters Call Stack
  |
  v
Route / Logic Executes
  |
  +--> If fast: response returned immediately
  |
  +--> If async I/O: delegated and callback returns later
  |
  v
HTTP Response
```

Demo: Slow Blocking Route

```
const http = require("http");

function blockCpu(seconds) {
  const start = Date.now();
  while (Date.now() - start < seconds * 1000) {}
}

const server = http.createServer((req, res) => {
  if (req.url === "/block") {
    blockCpu(5);
    res.end("Blocking task finished");
    return;
  }

  res.end("Normal response");
});
```

```
server.listen(3000, () => {  
  console.log("Server running on port 3000");  
});
```

Now open two browser tabs: 1. `/block` 2. `/`

Students will see that CPU blocking hurts other requests.

Demo: Non-blocking Route

```
const http = require("http");  
  
const server = http.createServer((req, res) => {  
  if (req.url === "/wait") {  
    setTimeout(() => {  
      res.end("Non-blocking wait finished");  
    }, 5000);  
    return;  
  }  
  
  res.end("Normal response");  
});  
  
server.listen(3000, () => {  
  console.log("Server running on port 3000");  
});
```

Now open: 1. `/wait` 2. `/`

Students will see that the server can still respond to normal requests.

4. Interview Preparation Section

Must-Know Interview Questions

1. What is Node.js?
2. Is Node.js single-threaded?
3. How can Node.js handle multiple requests?
4. What is the event loop?
5. What is the difference between synchronous and asynchronous?
6. What is the difference between blocking and non-blocking?
7. What is the difference between `require` and `module.exports`?
8. What is npm?
9. What is `package.json`?

10. What is the difference between dependencies and devDependencies?
 11. What is the HTTP module?
 12. Why do we use Express if Node.js already has an HTTP module?
 13. What happens internally when a request hits a Node.js server?
 14. Why should CPU-heavy work be avoided in the main thread?
 15. What is the difference between Node.js concurrency and true parallelism?
-

5. Final Lab

Students build a mini server with these routes:

```
GET /           -> Welcome message
GET /about      -> About message
GET /time       -> Current server time
GET /users?id=1 -> Parse query parameter using URL module
GET /block      -> Demonstrate blocking behavior
GET /wait       -> Demonstrate non-blocking behavior
```

Expected skills:

- Use `http` module
 - Use `url` module
 - Use routing with `req.url`
 - Understand request/response flow
 - Observe blocking vs non-blocking behavior
-

6. Homework

1. Create a custom module named `responseHelper.js`.
2. Export a function called `sendJson`.
3. Use it inside the HTTP server.
4. Add routes:
5. `/products`
6. `/orders`
7. `/health`
8. Add npm scripts:
9. `start`

10. `dev`

11. Write answers to 10 interview questions from today's class.

7. One-Line Summary for Students

Node.js is powerful not because JavaScript becomes multithreaded, but because Node.js uses an event-driven, non-blocking architecture to keep the main thread free while slow tasks complete outside the main execution flow.